

WebSocket 通信接口定义 -- Ver2.2

背景

本 Demo 原计划采用 wholebody VLA 端到端方案——由单一模型同时输出上肢抓取动作与底盘移动指令。然而 wholebody 方案需要采集全身协同的示教数据，数据采集工作量极大，两周内难以完成。

为此改为分段式方案：由灵动科技负责底盘移动与 AMR 调度，阿加犀 VLA 模型仅控制上肢完成抓取与放置。底盘与上肢之间通过 WebSocket 指令衔接——上肢完成一段动作后通知底盘，底盘移动到位后通知上肢继续。

分段式方案对**定位精度**提出了硬性要求：

- TRON2 底盘在桌面侧与料架侧之间的停靠偏差需控制在 **厘米级 ($\leq \pm 5\text{cm}$)**
- 货架 AMR 在交互工位的停靠偏差需控制在 **厘米级 ($\leq \pm 5\text{cm}$)**
- 底盘与 AMR 停靠时的**角度误差 $\leq 5^\circ$** ，确保上肢末端可达范围内料箱朝向一致

若底盘/AMR 停靠偏差过大，上肢预设的抓取/放置位姿将失效，需重新示教或由视觉补偿。

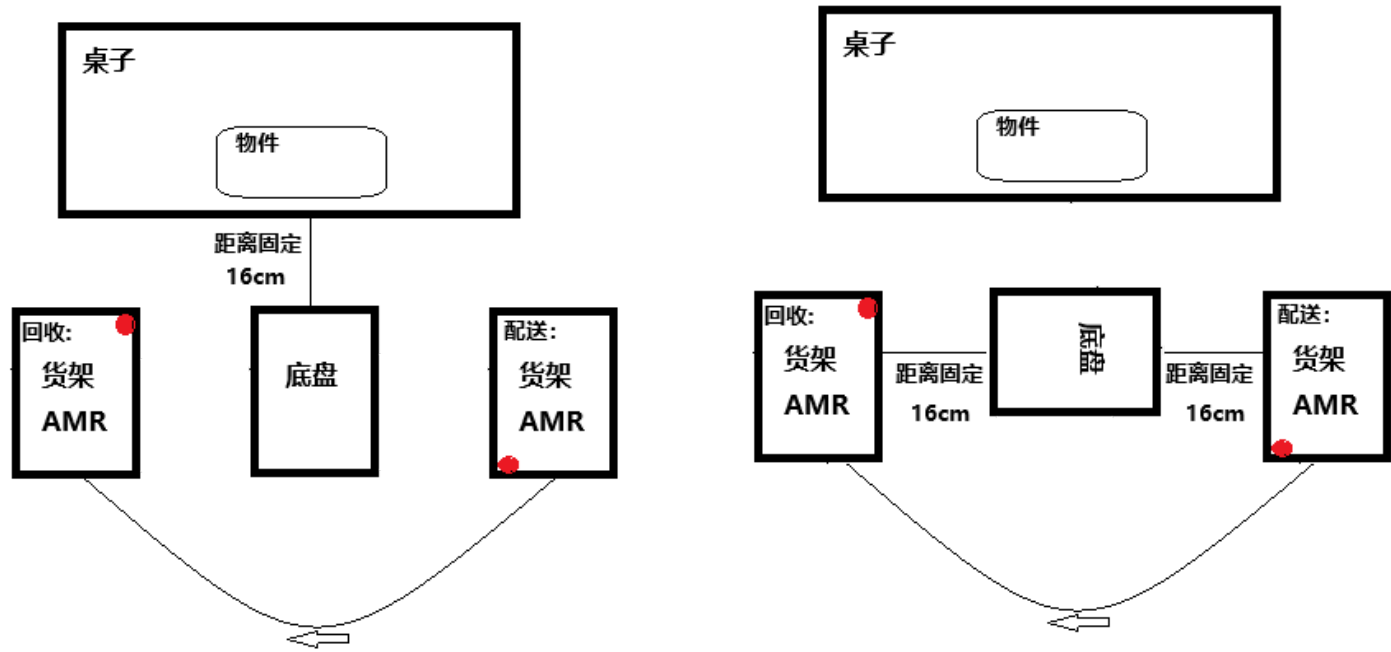
一、分工原则

负责方	设备	职责
阿加犀	TRON2 上身 (设备1)	VLA 推理控制双臂完成抓取与放置；接收底盘指令、上报状态
灵动科技	底盘 (设备2) + 货架 AMR (设备3) + Laptop(设备) 的控制/演示	底盘移动/旋转、AMR 环形运行——自主闭环，到位后通知上肢； Laptop UI: 全局控制 (start_demo / stop / status)

阿加犀只控手，灵动控脚和车。上肢不关心底盘怎么动、AMR 怎么绕。

二、环境和通信拓扑

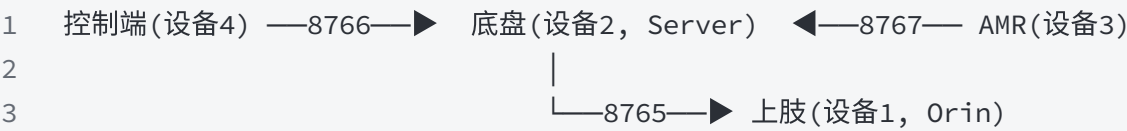
2.1 场景布置



2.2 通信拓扑

底盘（设备2）作为 WebSocket **Server**，也是整个 Demo 的中控枢纽。其余设备各自作为 **Client** 连接。

代码块



端口	Server	Client	用途
----	--------	--------	----

8765	底盘	上肢 (Orin)	任务指令（run / stop / status）与状态回报（done）
8766	底盘	演示控制端	全局控制（start_demo / stop / status）
8767	底盘	货架 AMR	AMR 到位通知（arrived）与放行（go）

> 底盘作为唯一中控，所有消息直达，**Orin 无需转发任何消息。**

三、接口 A：底盘 ↔ 上肢（端口 8765）

底盘为 Server，上肢 (Orin) 为 Client。底盘通过此端口下发任务指令并接收状态。

3.1 任务模型

Demo 演示搬运 4 件物体。每件物体的"拿 + 放"拆成两段，由底盘分两次下发不同 `task`，循环 4 次：

代码块

```
1  循环 ×4：
2    底盘发 run_grasp_from_table → 上肢抓取 → 上肢回 done
3    → 底盘移动换位（桌面→料架侧）
4    → 底盘发 run_place_to_shelf → 上肢放置 → 上肢回 done
5    → 底盘移动换位（料架→桌面侧，准备下一件）
```

任务从命名上直接区分动作，上肢无需感知自身位置：

task	动作	说明
<code>grasp_from_table</code>	抓取	从桌面料箱抓取 1 件物体
<code>place_to_shelf</code>	放置	将手中物体放入货架 AMR 料箱

grasp_from_shelf	抓取	从货架 AMR 料箱抓取 1 件物体
place_to_table	放置	将手中物体放回桌面料箱

配送阶段：grasp_from_table + place_to_shelf 循环 4 次，AMR 全程停靠等待。全部放置完成后 AMR 离开绕圈。

回收阶段：AMR 返回后，grasp_from_shelf + place_to_table 循环 4 次，全部回收完成后 AMR 离开。

3.2 底盘 → 上肢（Command）

启动任务

代码块

```
1  {
2      "cmd": "run",
3      "task": "grasp_from_table"
4  }
```

字段	类型	说明
cmd	string	固定 "run"
task	string	grasp_from_table / place_to_shelf / grasp_from_shelf / place_to_table

停止任务

代码块

```
1  {"cmd": "stop"}
```

收到后上肢立即终止当前动作，回到安全姿态，回复 `stopped`。

查询状态

代码块

```
1  {"cmd": "status"}
```

上肢收到后立即回复一条当前 `status` 消息。

3.3 上肢 → 底盘（Response）

运行中（周期性推送）

代码块

```
1  {
2      "type": "status",
3      "task": "grasp_from_table",
4      "running": true,
5      "step": 12,
6      "message": "model running"
7  }
```

字段	类型	说明
<code>type</code>	string	固定 <code>"status"</code>
<code>task</code>	string	当前任务类型
<code>running</code>	bool	<code>true</code> 表示模型正在推理执行
<code>step</code>	int	当前推理步数

message	string	状态描述
---------	--------	------

推送频率：VLA 推理帧率（~10–50 Hz， 30 Hz Default）。

任务完成

代码块

```
1  {
2      "type": "done",
3      "task": "grasp_from_table",
4      "running": false,
5      "success": true,
6      "message": "搬运完成"
7  }
```

字段	说明
type	固定 "done"
success	true 表示当前段（抓取/放置）成功完成

重要：上肢发送 done 后即退出控制，等待底盘移动到下一位置后再次下发 run。

任务被停止

代码块

```
1  {
2      "type": "stopped",
3      "task": "grasp_from_table",
4      "running": false,
5      "success": false,
6      "message": "任务已停止"
7  }
```

```
7 }
```

执行出错 (可选)

代码块

```
1 {
2     "type": "error",
3     "task": "grasp_from_table",
4     "running": false,
5     "success": false,
6     "error": "具体错误信息",
7     "message": "任务执行失败"
8 }
```

字段	说明
error	具体错误描述（供调试）

四、接口 B：全局控制（端口 8766）

演示控制端（笔记本/平板，设备4）连接此端口下发全局指令。与 8765 的任务指令分离，保证急停等高优先级命令不受任务消息阻塞。

消息定义

急停

代码块

```
1 {"cmd": "stop"}
```

底盘收到后立即刹停，同时通过 8765 向上肢下发 stop ，上肢终止动作回安全姿态。

启动一轮 Demo

代码块

```
1 {"cmd": "start_demo"}
```

底盘收到后进入就绪，开始按流程通过 8765 向上肢下发 run 指令。

查询状态

代码块

```
1 {"cmd": "status"}
```

上肢收到后通过 8765 回复一条当前 status 消息。

速查

消息	说明
<code>{"cmd": "stop"}</code>	全局急停（上肢 + 底盘）
<code>{"cmd": "start_demo"}</code>	启动一轮 Demo，上肢进入就绪
<code>{"cmd": "status"}</code>	查询当前状态

五、接口 C：AMR ↔ 底盘（端口 8767）

货架 AMR 连接此端口进行到位握手与放行。

消息定义

AMR → 底盘：到位通知

代码块

```
1  {"type": "arrived"}
```

AMR 到达交互工位并停稳后发送。底盘收到后开始配送或回收循环。

底盘 → AMR：放行

代码块

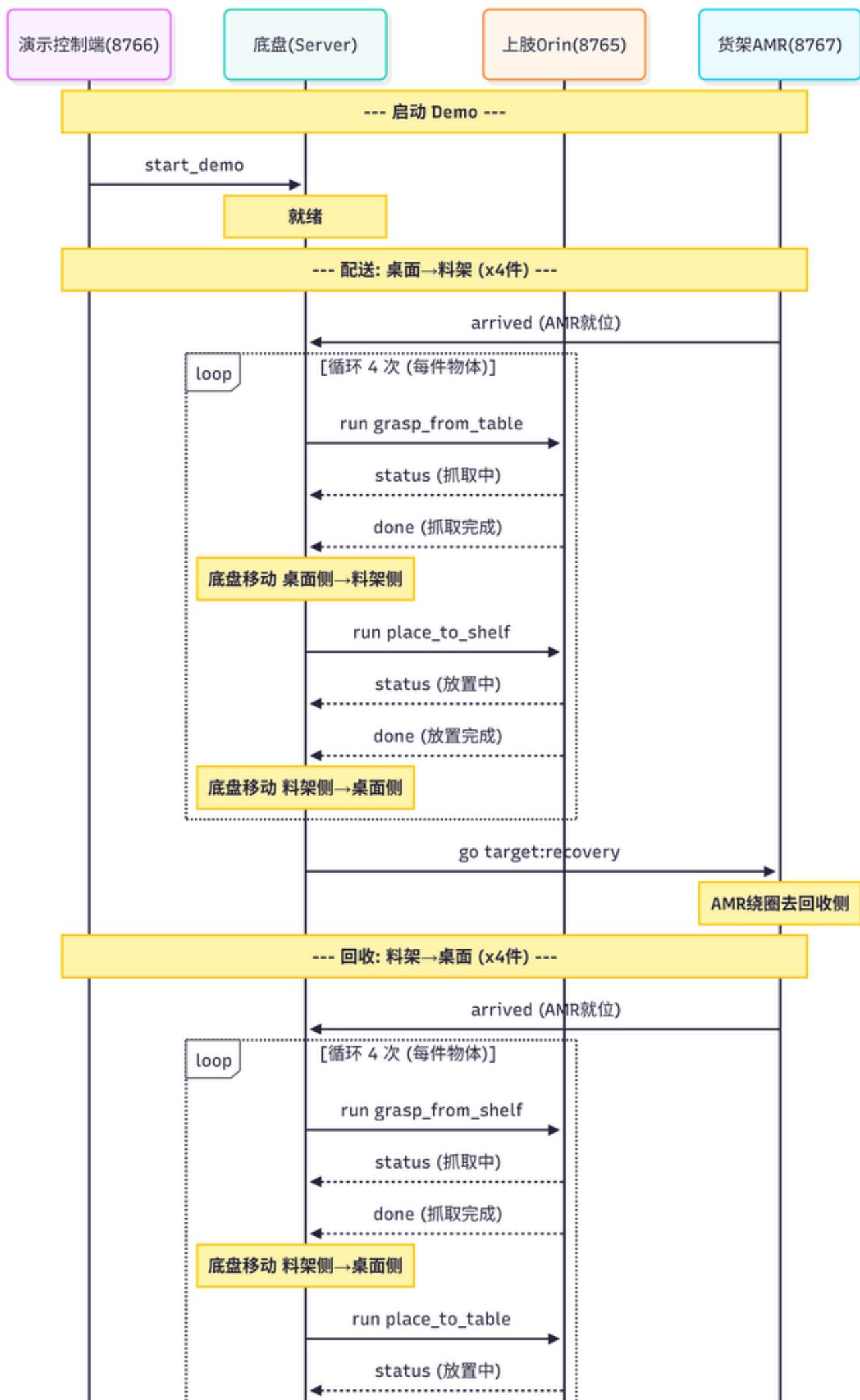
```
1  {"type": "go", "target": "recovery"}
```

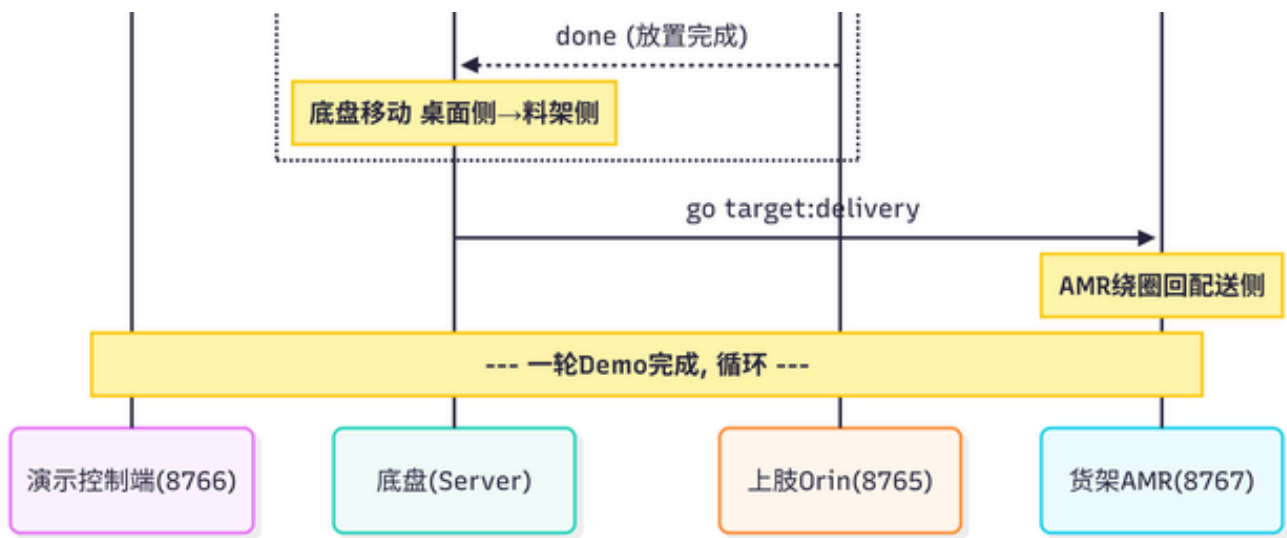
字段	类型	说明
type	string	固定 "go"
target	string	"recovery" — 去回收区； "delivery" — 去配送区

配送阶段 4 件放置完成后，底盘发 target: "recovery" 通知 AMR 绕圈去回收侧。回收阶段 4 件回收完成后，发 target: "delivery" 通知 AMR 绕圈回配送侧。

注：AMR 的环形运行调度由灵动科技 Fleet 系统管理，WebSocket 仅用于到位握手与放行。

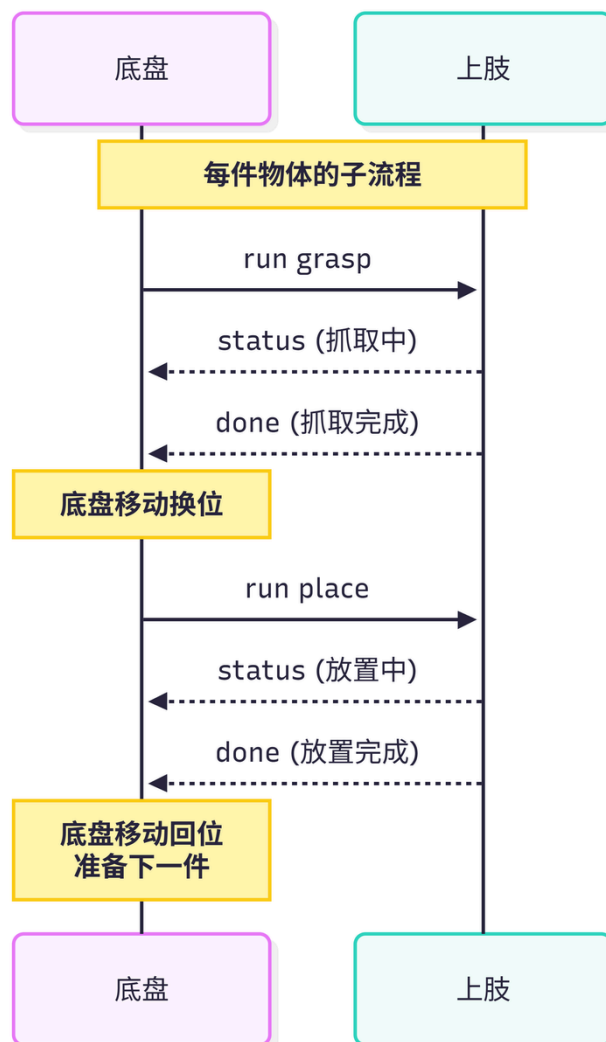
六、完整交互流程





6.1 单件搬运子流程（循环 ×4）

每件物体搬运的底盘↔上肢交互子流程如下，配送和回收阶段各循环 4 次：



步骤	发起方	消息	说明
①	底盘	run_grasp	从当前侧料箱抓取 1 件物体
②	上肢	done	抓取完成，底盘接管
—	底盘	(自主移动)	底盘旋转/移动至目标侧
③	底盘	run_place	将物体放入目标侧料箱
④	上肢	done	放置完成
—	底盘	(自主移动)	底盘移回抓取侧，准备下一件

配送阶段 task = grasp_from_table / place_to_shelf，回收阶段 task = grasp_from_shelf / place_to_table。各自循环 4 次。

6.2 AMR 参与时机

AMR 环形运行由灵动科技 Fleet 调度。每轮 Demo 中 AMR 停靠两次：

第一次停靠（配送）：

- AMR 到达料架交互位 → 发 arrived 给上肢
- 上肢告知底盘"AMR 已就位"
- 底盘在桌面侧与料架侧之间来回移动，上肢执行 grasp→place 子流程 ×4 次
- 4 件全部放置完成后，AMR 离开继续环形运行

第二次停靠（回收）：

- AMR 绕圈后再次到达交互位 → 发 arrived 给上肢
- 底盘在料架侧与桌面侧之间来回移动，上肢执行 grasp→place 子流程 ×4 次
- 4 件全部回收完成后，AMR 离开

AMR 在每次停靠期间保持静止，直到该阶段全部 4 件搬运完成。

6.3 异常处理

- 上肢执行中收到 `stop` → 终止动作，回 `stopped` ，底盘决定下一步
- 上肢执行出错 → 回 `error` ，底盘决定重试或跳过
- 底盘超时未收到 `done` （如 >60s）→ 可发 `status` 查询，或发 `stop` 终止

七、消息速查

底盘 → 上肢 (`:8765`)

消息	说明
<code>{"cmd":"run","task":"grasp_from_table"}</code>	从桌面抓取
<code>{"cmd":"run","task":"place_to_shelf"}</code>	放置到货架 AMR
<code>{"cmd":"run","task":"grasp_from_shelf"}</code>	从货架 AMR 抓取
<code>{"cmd":"run","task":"place_to_table"}</code>	放置到桌面
<code>{"cmd":"stop"}</code>	停止当前任务
<code>{"cmd":"status"}</code>	查询当前状态

上肢 → 底盘 (`:8765`)

消息	说明
<code>{"type":"status","task":"...","running":true,"step":N,"message":"..."}</code>	运行中（周期性）
<code>{"type":"done","task":"...","running":false,"success":true,"message":"..."}</code>	当前段完成

<pre>{"type":"stopped","task":"...","running":false,"success":false,"message":"..."}</pre>	已停止
<pre>{"type":"error","task":"...","running":false,"success":false,"error":"...","message":"..."}</pre>	执行出错

全局控制 (⌨:8766⌨)

消息	说明
<pre>{"cmd":"stop"}</pre>	全局急停
<pre>{"cmd":"start_demo"}</pre>	启动一轮 Demo
<pre>{"cmd":"status"}</pre>	查询当前状态

AMR ↔ 底盘 (⌨:8767⌨)

消息	方向	说明
<pre>{"type":"arrived"}</pre>	AMR → 底盘	AMR 到达交互位并停稳
<pre>{"type":"go","target":"recovery"}</pre>	底盘 → AMR	通知 AMR 绕圈去回收侧
<pre>{"type":"go","target":"delivery"}</pre>	底盘 → AMR	通知 AMR 绕圈回配送侧

八、联调步骤

1. 连接测试：底盘连 `ws://{Orin_IP}:8765`，演示控制端连 `ws://{Orin_IP}:8766`，AMR 连 `ws://{Orin_IP}:8767`，确认三条连接均成功。
2. 急停测试：演示控制端通过 `8766` 发 `{"cmd":"stop"}`，上肢应回 `stopped` 并复位，底盘应同步刹停。
3. Demo 启动：演示控制端通过 `8766` 发 `{"cmd":"start_demo"}`，上肢进入就绪。
4. 状态查询：底盘发 `{"cmd":"status"}` (`8765`)，上肢应回复当前 `idle` 状态。
5. 单件子流程（桌面→料架）：底盘依次发 `grasp_from_table` → 移动 → `place_to_shelf`，观察上肢抓取与放置各回复 `done`。

6. 配送循环 (×4)：底盘在桌面侧与料架侧之间来回移动 4 次，每次执行步骤 5 的子流程，确认 4 件全部放置到 AMR。
7. AMR 联动：AMR 到达发 `arrived` (`8767`)，上肢通知底盘，配送循环开始；4 件放置完成后 AMR 离开绕圈。
8. 回收循环 (×4)：AMR 绕圈返回再次发 `arrived`，底盘执行 `grasp_from_shelf` → 移动 → `place_to_table` 子流程 ×4 次。
9. 完整 Demo 循环：走通 `start_demo` → 配送×4 → AMR绕圈 → 回收×4 一整轮。